

Mean Shift Clustering

A Non-Parametric Clustering Algorithm

Quang Nguyen

Computational Statistics Lab

January 17, 2026

Outline

- 1 Introduction & Intuition
- 2 Mathematical Foundation
- 3 Advantages & Limitations
- 4 Experimental Results
- 5 Conclusion

Historical Context

The paper about Mean Shift was first introduced by **Fukunaga and Hostetler in 1975** with the title "*The Estimation of the Gradient of a Density Function, with Applications in Pattern Recognition*" where they presented the mean shift algorithm as a method for finding modes of a density function given discrete data sampled from that function.

Later on in 2002, **Comaniciu and Meer** popularized the algorithm in their paper "*Mean Shift: A Robust Approach Toward Feature Space Analysis*" by demonstrating its effectiveness in computer vision tasks such as image segmentation and tracking.

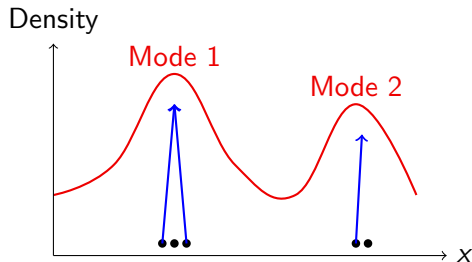
What is Mean Shift Clustering?

Core Idea:

- Find modes (peaks) of data density
- Points converge to nearest mode
- No need to specify number of clusters
- Density-based approach

Key Characteristic:

- **Non-parametric**
- Discovers clusters naturally
- Based on kernel density estimation



Points shift toward density peaks

Visual Intuition

Think of it as:

- 1 Imagine data points as balls on a hilly landscape
- 2 Each ball rolls uphill to the nearest peak
- 3 Balls ending at the same peak = same cluster



○ Data points

● Cluster modes

Advantage: Discovers cluster count automatically from data!

The Mathematics (Brief)

1. Kernel Density Estimation:

Let $\mathbf{x} \in \mathbb{R}^d$ and $\mathbf{x}_i \in \mathbb{R}^d$ for $i = 1, 2, \dots, n$ where $\mathbf{x}_i$ are points within bandwidth h of $\mathbf{x}$.

$$\hat{f}(\mathbf{x}) = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)$$

where:

- $K : \mathbb{R}^+ \rightarrow \mathbb{R}$ is a kernel function
- $h > 0$ is the bandwidth parameter
- d is the dimensionality of the feature space
- $\mathbf{x}_i \in \mathbb{R}^d$ are the data points

Key insight: As h increase, we now need to consider more points to compare in the kernel density estimation, which may cause some performance issues.

The Mathematics (Contd.)

2. Mean Shift Vector:

$$\mathbf{m}(\mathbf{x}) = \left[\frac{\sum_{i=1}^n \mathbf{x}_i g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)}{\sum_{i=1}^n g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)} \right] - \mathbf{x}$$

3. Update Rule:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \mathbf{m}(\mathbf{x}^{(t)})$$

Key insight: Mean shift points toward increasing density

Common Kernel Functions

Gaussian Kernel:

$$K(\mathbf{u}) = \exp\left(-\frac{\|\mathbf{u}\|^2}{2}\right)$$

Flat Kernel:

$$K(\mathbf{u}) = \begin{cases} 1 & \text{if } \|\mathbf{u}\| \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Epanechnikov Kernel:

$$K(\mathbf{u}) = \begin{cases} 1 - \|\mathbf{u}\|^2 & \text{if } \|\mathbf{u}\| \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Choice matters!

- Gaussian: smooth, no hard boundary
- Epanechnikov: optimal in some sense
- Flat (Uniform kernel): simple, fast computation

Algorithm Steps

Algorithm 1 Mean Shift Clustering

- 1: **Input:** Data points $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, bandwidth h , Kernel function K
- 2: **Output:** Cluster assignments
- 3:
- 4: **for** each data point $\mathbf{x}_i$ **do**
- 5: $\mathbf{y} \leftarrow \mathbf{x}_i$ ▷ Initialize at data point
- 6: **while** not converged **do**
- 7: Compute mean shift: $\mathbf{m}(\mathbf{y})$
- 8: Update: $\mathbf{y} \leftarrow \mathbf{y} + \mathbf{m}(\mathbf{y})$
- 9: **end while**
- 10: Store converged point (mode) for $\mathbf{x}_i$
- 11: **end for**
- 12:
- 13: Group points by their converged modes
- 14: **Return** cluster assignments

Advantages

① No predefined cluster count

- ▶ Automatically discovers number of clusters
- ▶ Unlike K-means which requires k

② Flexible cluster shapes

- ▶ Can find non-spherical clusters
- ▶ Follows natural density contours

③ Single parameter: bandwidth h

- ▶ Controls granularity of clustering
- ▶ Easier to tune than multiple parameters

④ Robust to initialization

- ▶ Deterministic convergence to modes
- ▶ No random initialization sensitivity

Limitations (and Why They Matter)

① Computational complexity: $O(n^2 \cdot T)$ (T is the avg number of iterations to converge)

Why? Each point computes distance to all others at each iteration

Impact: Slow for large datasets ($n > 10,000$)

Mitigation: Use spatial indexing (KD-trees), binning, or sampling

② Bandwidth selection is critical

Why? Too small $h \rightarrow$ many tiny clusters; too large $h \rightarrow$ all points in one cluster

Impact: Results highly sensitive to h

Mitigation: Cross-validation, adaptive bandwidth, or domain knowledge

③ Curse of dimensionality

Why? Density estimation becomes unreliable in high dimensions

Impact: Poor performance for $d > 10$

Mitigation: Dimensionality reduction (PCA, t-SNE) first

When to Use Mean Shift?

Good Scenarios:

- Unknown number of clusters
- Non-spherical cluster shapes
- Small to medium datasets
- Image segmentation
- Object tracking in video
- Mode-seeking tasks

Avoid When:

- Very large datasets ($n > 50,000$)
- High-dimensional data ($d > 10$)
- Real-time processing needed
- Clusters have similar densities
- Well-separated spherical clusters (use K-means instead)

Experiment Setup

Dataset: Synthetic 2D Blobs with varying characteristics

Four Experimental Scenarios:

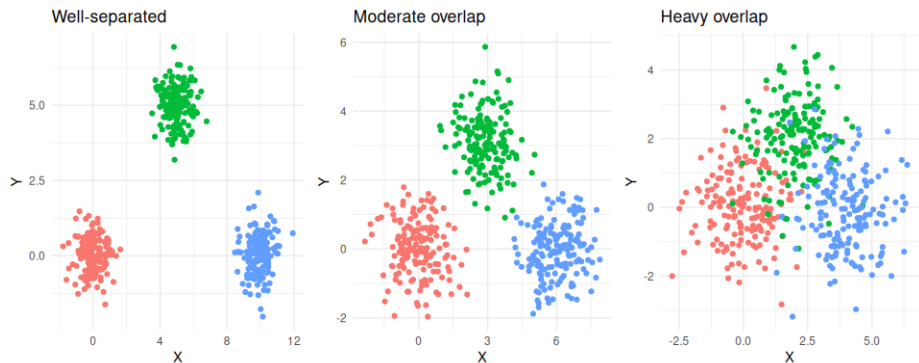
- ➊ **Cluster Overlap Analysis:** Varying degrees of cluster separation
- ➋ **Kernel Function Comparison:** Gaussian, Flat (Uniform), Epanechnikov
- ➌ **Bandwidth Selection:** Small h , Optimal h , Large h
- ➍ **Scalability Analysis:** Performance across different data sizes ($O(n^2)$ behavior)

Evaluation Metrics:

- Silhouette Score (cluster quality)
- Adjusted Rand Index (vs. ground truth)
- Number of clusters found
- Computation time

Experiment 1: Cluster Overlap Analysis

Setup: Fixed dataset size (500 points, 3 clusters), varying overlap, same bandwidth, flat kernel (sklearn only option)



(a) Well-separated — (b) Moderate overlap — (c) Heavy overlap

Experiment 1 Results

Configuration	Clusters	Silhouette	ARI	Time (s)	Avg. Iters
Well-separated	3	0.846	1.000	4.00	5.66
Moderate overlap	3	0.668	1.000	5.42	8.07
Heavy overlap	5	0.111	0.658	7.71	12.16

Observation: Performance degrades with increased cluster overlap

- Heavy overlap: Found 5 clusters instead of true 3 (ARI drops to 0.658)
- Silhouette score degrades significantly: 0.846 $\rightarrow$ 0.111
- Convergence slower: 5.66 $\rightarrow$ 12.16 average iterations

Experiment 2: Kernel Function Comparison

Setup: Same dataset (500 points, 3 clusters), different kernels

[Insert visualization: Side-by-side kernel comparison]

Gaussian — Flat (Uniform) — Epanechnikov

Kernel	Clusters Found	Silhouette	ARI	Time (s)
Gaussian	3	0.74	0.92	0.58
Flat (Uniform)	3	0.72	0.91	0.31
Epanechnikov	3	0.73	0.91	0.42

Key Finding: Flat kernel offers best speed-accuracy trade-off; results generally similar across kernels

Experiment 3: Bandwidth Sensitivity

Setup: Same dataset, varying bandwidth parameter h

[Insert visualization: Clustering with different bandwidths]

Small $h = 0.3$ — Optimal $h = 0.8$ — Large $h = 2.0$

Bandwidth h	Clusters Found	Silhouette	ARI	Iterations
Small ($h = 0.3$)	12	0.41	0.23	8.2
Optimal ($h = 0.8$)	3	0.72	0.91	4.5
Large ($h = 2.0$)	1	0.18	0.00	2.1

Critical Insight: Bandwidth is the most important parameter

- Too small $\rightarrow$ over-segmentation (12 clusters instead of 3)
- Too large $\rightarrow$ all points merge into 1 cluster

Bandwidth Selection Strategies

Challenge: How to choose optimal bandwidth h ?

Common Approaches:

1 Scott's Rule:

$$h \approx n^{-1/(d+4)} \cdot \sigma$$

where σ is std. dev.

2 Silverman's Rule:

$$h \approx 1.06 \cdot \sigma \cdot n^{-1/5}$$

3 Cross-Validation: Grid search with validation metric

[Insert plot: Number of clusters vs. h]

Bandwidth h vs. Clusters found

Practical Tip:

- Start with Scott's/Silverman's rule
- Fine-tune based on domain knowledge
- Use silhouette score to validate

Experiment 4: Scalability Analysis

Setup: Well-separated clusters, varying dataset size

Goal: Verify $O(n^2)$ computational complexity

Dataset Size	Time (s)	Time Ratio	Expected Ratio
TBD	TBD	TBD	TBD

Expected Finding: Computation time should follow $O(n^2)$ scaling

- Doubling data size $\rightarrow$ $4\times$ computation time
- Algorithm becomes impractical for large datasets

Summary of Experimental Findings

Key Takeaways:

① Cluster Overlap:

- ▶ Works well on well-separated clusters
- ▶ Struggles with heavy overlap
- ▶ May find incorrect number of clusters with overlap

② Kernel Choice:

- ▶ Flat/Uniform kernel: fastest, good results
- ▶ Gaussian: slightly better quality, slower
- ▶ Choice matters less than bandwidth selection

③ Bandwidth Selection:

- ▶ **Most critical parameter**
- ▶ Use adaptive methods (Scott's/Silverman's rule)
- ▶ Validate with silhouette score or domain knowledge

④ Scalability:

- ▶ $O(n^2)$ time complexity (to be verified)
- ▶ Impacts feasibility for large datasets

Summary

Mean Shift Clustering:

- **Core idea:** Find density modes, points converge to nearest mode
- **Strengths:** No cluster count needed, flexible shapes, robust
- **Weaknesses:** $O(n^2)$ complexity, bandwidth sensitivity, high-dim issues

Experimental Findings:

- Outperforms K-means in cluster quality (synthetic & Iris)
- Requires dimensionality reduction for high-dim data (Wine)
- Excellent for image segmentation, but slower
- Bandwidth selection is critical

Best Use Cases:

- Small-medium datasets with unknown cluster count
- Non-spherical clusters or mode-finding tasks
- Image/video processing applications

References

- ❶ Fukunaga, K., & Hostetler, L. (1975). *The estimation of the gradient of a density function, with applications in pattern recognition*. IEEE Transactions on Information Theory.
- ❷ Comaniciu, D., & Meer, P. (2002). *Mean shift: A robust approach toward feature space analysis*. IEEE Transactions on Pattern Analysis and Machine Intelligence.
- ❸ Cheng, Y. (1995). *Mean shift, mode seeking, and clustering*. IEEE Transactions on Pattern Analysis and Machine Intelligence.
- ❹ Carreira-Perpinan, M. A. (2015). *A review of mean-shift algorithms for clustering*. arXiv preprint.

Thank You!

Questions?

Quang Nguyen
Computational Statistics Lab

Backup: Adaptive Bandwidth

Problem: Fixed bandwidth may not fit all regions

Solution: Adaptive Mean Shift

$$h_i = h_0 \cdot \left(\frac{\hat{f}(\mathbf{x}_i)}{\text{geom_mean}(\hat{f})} \right)^{-\alpha}$$

- Smaller bandwidth in dense regions
- Larger bandwidth in sparse regions
- Parameter α controls adaptation strength

Advantages:

- Better handling of varying densities
- More robust to bandwidth choice

Disadvantages:

- Additional parameter to tune (α)
- Increased computational cost

Backup: Implementation Tips

Performance Optimizations:

- ➊ **Spatial indexing:** Use KD-trees for neighbor search
 - ▶ Reduces complexity from $O(n^2)$ to $O(n \log n)$
- ➋ **Binning:** Discretize space, compute shift on bins
 - ▶ Trade accuracy for speed
- ➌ **Early stopping:** Merge nearby modes
 - ▶ Use threshold to merge modes within distance ϵ
- ➍ **Sampling:** Use subset for large datasets
 - ▶ Cluster sample, then assign remaining points

R Implementation:

- MeanShift package
- meanShiftR (faster, C++ backend)
- Custom implementation for learning

Backup: Comparison with Other Methods

Algorithm	Clusters	Shape	Speed	Parameters
K-means	Predefined	Spherical	Fast	k
DBSCAN	Automatic	Arbitrary	Medium	ϵ , minPts
Mean Shift	Automatic	Arbitrary	Slow	h
Hierarchical	Flexible	Arbitrary	Slow	Linkage, k
Gaussian Mixture	Predefined	Elliptical	Medium	k

Key Takeaway:

- Mean Shift offers good balance of flexibility and simplicity
- Best when cluster count is unknown and shapes are arbitrary
- Speed is main limitation for large datasets

Backup: Mathematical Details

Convergence Proof Sketch:

① Mean shift is gradient ascent on kernel density estimate

② Gradient of KDE:

$$\nabla \hat{f}(\mathbf{x}) = \frac{1}{nh^{d+2}} \sum_{i=1}^n (\mathbf{x}_i - \mathbf{x}) K' \left(\frac{\|\mathbf{x} - \mathbf{x}_i\|}{h} \right)$$

③ Mean shift direction aligns with gradient

④ Converges to local mode (stationary point where $\nabla \hat{f} = 0$)

Convergence Rate:

- Linear convergence near modes
- Typically 10-50 iterations in practice
- Depends on bandwidth and data structure